AI Interview Mentor

A Multimodal RAG-Centered Interview Preparation System

Graduation Project Concept Document — Prepared for team review and academic supervisor discussion.

1. Project Overview

AI Interview Mentor is a multimodal platform that helps a candidate prepare for a real job interview. The system runs a full Mock Interview and evaluates the candidate's performance based on the specific job description and company they are applying to — not generic questions.

Unlike a typical chatbot-over-documents demo, **Retrieval-Augmented Generation (RAG) is the core engine of the entire system**. Computer Vision (CV) and Natural Language Processing (NLP) are supporting modules that feed signals into the RAG pipeline, rather than standalone components.

Target User / Use Case

A candidate with an upcoming interview, for example: *"I have a Backend Python interview at Valeo in two days."* The system builds a personalized preparation plan and interview simulation around this exact context.

2. Phase 1 — Knowledge Base Construction (RAG Foundation)

The user provides the following inputs:

- Resume / CV
- Job Description
- Company name
- Company documents (if available)
- GitHub profile (optional)

What the RAG pipeline does with each input:

Resume: Parsing → Chunking → Embedding

Job Description: Chunking → Embedding

Company Information: Aggregated data: what the company does, tech stack used, company culture, past/ongoing projects

Interview Questions Database: Thousands of curated questions across Backend, Frontend, AI, Data Science, and HR tracks

HR Evaluation Guidelines: Structured evaluation criteria: Communication, Leadership, Problem Solving, Technical Skills

All processed content is embedded and stored in a Vector Database. This is where the RAG system effectively begins operating.

3. Phase 2 — Candidate Gap Analysis

The RAG engine compares the Resume against the Job Description to produce a structured skills gap report:

- Skills the candidate already has
- Skills that are missing
- Critical skills the candidate must study before the interview

4. Phase 3 — Personalized Study Plan Generation

Instead of generic advice, the system grounds recommendations in retrieved content. For example, if the gap analysis flags **Redis** as missing:

- Retrieves relevant official documentation
- Retrieves top tutorials
- Retrieves related interview questions
- Builds a personalized study roadmap

5. Phase 4 — Mock Interview (Retrieval-Driven Questioning)

This is the core of the system. The AI does **not** generate interview questions freely from the LLM. Instead, it performs retrieval across:

- The Interview Questions Database
- Company-specific questions
- The Job Description
- The candidate's Resume

It then selects the most relevant question for that specific candidate and role. For example, because the candidate listed FastAPI on their resume and the job requires Docker knowledge, the system asks:

> *"Explain Dependency Injection in FastAPI."*

...rather than a generic, unrelated question such as "What is a CNN?"

6. Phase 5 — Capturing the Candidate's Answer

The candidate answers by voice. The audio pipeline is:

```
Voice → Speech-to-Text → Transcript
```

*Note: Speech-to-Text is purely an audio-to-text conversion step. It is **not** part of the NLP analysis itself.*

NLP Module

Once the transcript is produced, the NLP module analyzes:

- Grammar
- Fluency
- Keyword coverage
- Technical correctness
- Repeated / filler words
- Confidence in language use

Computer Vision Module

If the camera is enabled, the CV module analyzes:

- Eye contact
- Smiling
- Facial emotion
- Head pose
- Looking away from camera
- General body language

7. Phase 6 — RAG-Based Answer Evaluation

After the candidate answers, the LLM does **not** judge the answer from its own internal knowledge alone. It performs a second retrieval step, pulling in:

- The ideal/reference answer
- HR evaluation criteria
- Job requirements
- The candidate's existing experience from their resume

It then compares the candidate's answer against these retrieved references. For example, if the candidate answers a Docker question, the system retrieves Docker documentation and checks whether the answer covered key concepts such as Containers, Images, Volumes, and Networking. It might conclude:

> *"Your answer is good, but incomplete — it is missing volumes and networking."*

8. Phase 7 — Final Interview Report

After the mock interview ends, the RAG engine compiles a full performance report:

Category	Score / Result
Technical Score	85% (17 of 20 questions answered correctly)
Communication Score	78%
Body Language Score	82%

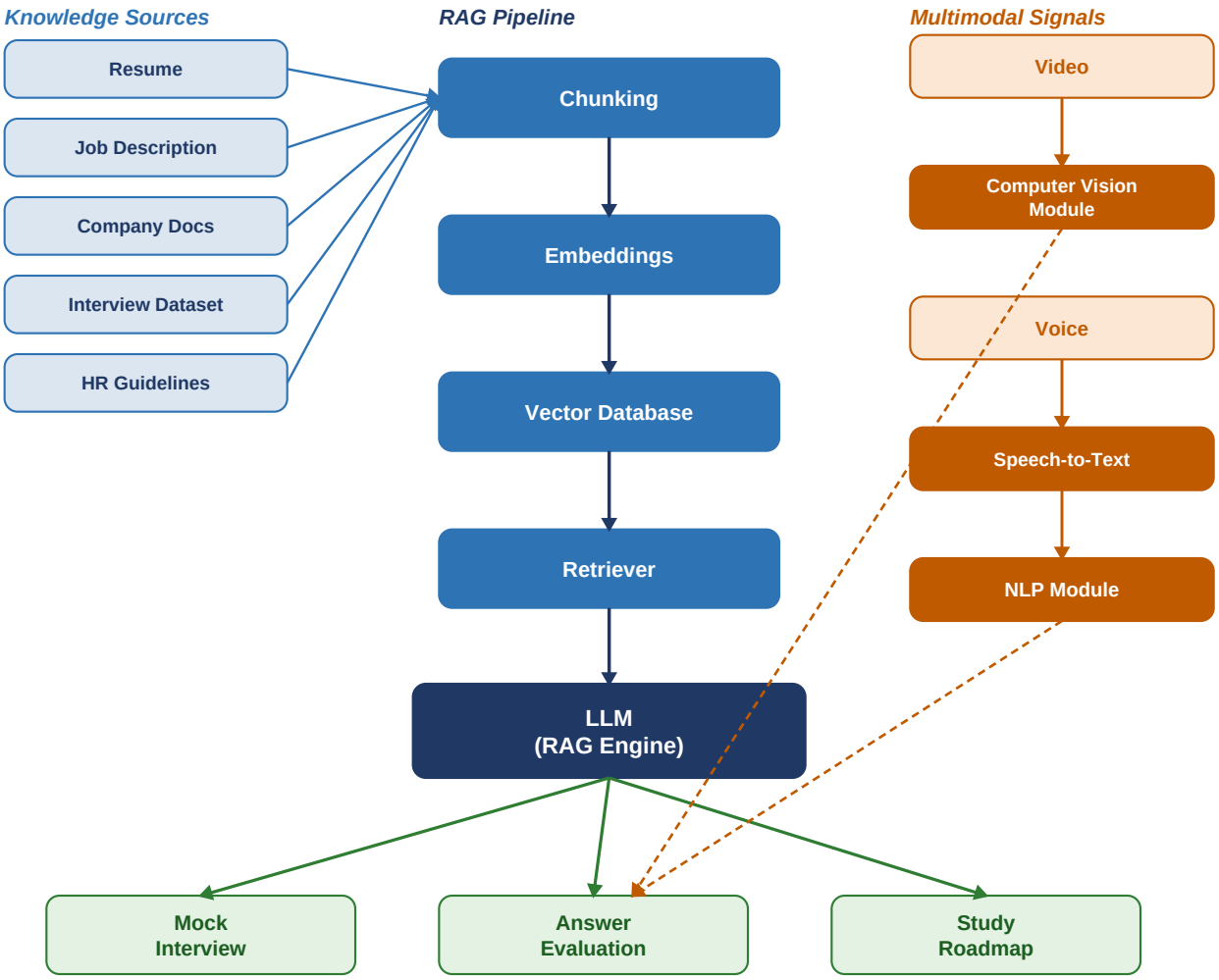
Weak Topics: Redis, Kubernetes, System Design

Strengths: FastAPI, Python, MongoDB

Recommended Study Plan: generated directly from the Knowledge Base (retrieved documentation, tutorials, and targeted questions).

9. System Architecture

The diagram below shows how the knowledge sources feed the RAG pipeline, how the LLM produces the three core outputs, and how the multimodal (voice/video) signals feed into the evaluation step.



10. Component Weight Breakdown

Component	Weight	Responsibilities
RAG (Core Engine)	70%	Knowledge base construction, question retrieval, resume-to-job comparison, reference-base
NLP	20%	Speech-to-Text (supporting tool), transcript analysis, answer quality scoring, keyword extrac
Computer Vision	10%	Body language analysis, eye contact tracking, facial expression analysis.

11. Proposed Enhancement: Multi-RAG Architecture

To strengthen the RAG component even further — and to make it clear during discussion that RAG is an architectural core of the system rather than a simple "chat with your files" feature — the single Knowledge Base can be split into four separate, purpose-built knowledge bases:

1. **Resume KB** — Everything related to the candidate (skills, experience, projects).
2. **Job & Company KB** — Job description, required skills, company profile and culture.
3. **Interview KB** — Interview questions, model answers, evaluation rubrics.
4. **Learning KB** — Courses, official documentation, and study roadmaps.

When the user makes a request, the system dynamically selects the most relevant knowledge base — or combines several of them — to generate the response. This routing behavior closely resembles a true **Multi-RAG** architecture, which is a strong point to highlight during the project defense, since it demonstrates that RAG is a core architectural component of the system, not just a chatbot layered on top of documents.